

simplest partitioner involves computing the hash value of the key and then taking the mod of that value with the number of reducers. This assigns approximately the same number of keys to each reducer (dependent on the quality of the hash function). Note, however, that the partitioner only considers the key and ignores the value—therefore, a roughly-even partitioning of the key space may nevertheless yield large differences in the number of key-values pairs sent to each reducer (since different keys may have different numbers of associated values). This imbalance in the amount of data associated with each key is relatively common in many text processing applications due to the Zipfian distribution of word occurrences.

Combiners are an optimization in MapReduce that allow for local aggregation before the shuffle and sort phase. We can motivate the need for combiners by considering the word count algorithm in Figure 2.3, which emits a key-value pair for each word in the collection. Furthermore, all these key-value pairs need to be copied across the network, and so the amount of intermediate data will be larger than the input collection itself. This is clearly inefficient. One solution is to perform local aggregation on the output of each mapper, i.e., to compute a local count for a word over all the documents processed by the mapper. With this modification (assuming the maximum amount of local aggregation possible), the number of intermediate key-value pairs will be at most the number of unique words in the collection times the number of mappers (and typically far smaller because each mapper may not encounter every word).

The combiner in MapReduce supports such an optimization. One can think of combiners as “mini-reducers” that take place on the output of the mappers, prior to the shuffle and sort phase. Each combiner operates in isolation and therefore does not have access to intermediate output from other mappers. The combiner is provided keys and values associated with each key (the same types as the mapper output keys and values). Critically, one cannot assume that a combiner will have the opportunity to process *all* values associated with the same key. The combiner can emit any number of key-value pairs, but the keys and values must be of the same type as the mapper output (same as the reducer input).¹² In cases where an operation is both associative and commutative (e.g., addition or multiplication), reducers can directly serve as combiners. In general, however, reducers and combiners are not interchangeable.

In many cases, proper use of combiners can spell the difference between an impractical algorithm and an efficient algorithm. This topic will be discussed in Section 3.1, which focuses on various techniques for local aggregation. It suffices to say for now that

¹²A note on the implementation of combiners in Hadoop: by default, the execution framework reserves the right to use combiners at its discretion. In reality, this means that a combiner may be invoked zero, one, or multiple times. In addition, combiners in Hadoop may actually be invoked in the reduce phase, i.e., after key-value pairs have been copied over to the reducer, but before the user reducer code runs. As a result, combiners must be carefully written so that they can be executed in these different environments. Section 3.1.2 discusses this in more detail.

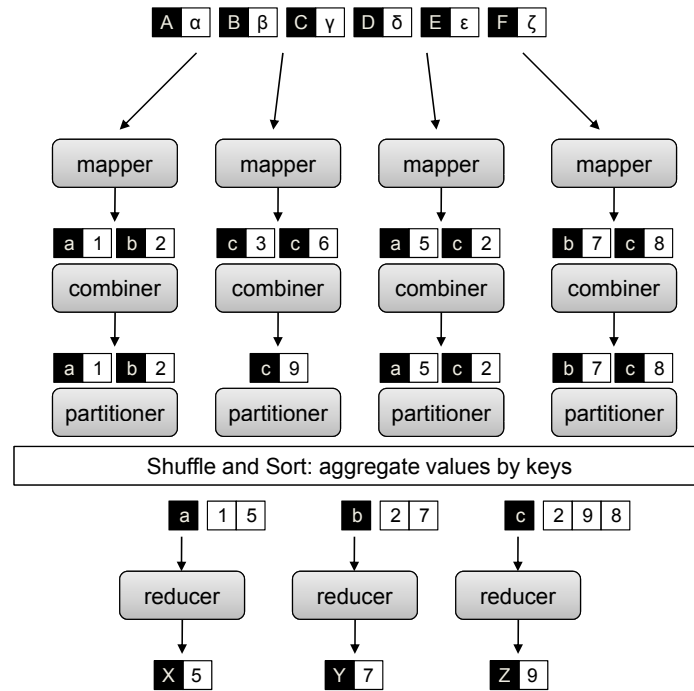


Figure 2.4: Complete view of MapReduce, illustrating combiners and partitioners in addition to mappers and reducers. Combiners can be viewed as “mini-reducers” in the map phase. Partitioners determine which reducer is responsible for a particular key.

a combiner can significantly reduce the amount of data that needs to be copied over the network, resulting in much faster algorithms.

The complete MapReduce model is shown in Figure 2.4. Output of the mappers are processed by the combiners, which perform local aggregation to cut down on the number of intermediate key-value pairs. The partitioner determines which reducer will be responsible for processing a particular key, and the execution framework uses this information to copy the data to the right location during the shuffle and sort phase.¹³ Therefore, a complete MapReduce job consists of code for the mapper, reducer, combiner, and partitioner, along with job configuration parameters. The execution framework handles everything else.

¹³In Hadoop, partitioners are actually executed before combiners, so while Figure 2.4 is conceptually accurate, it doesn’t precisely describe the Hadoop implementation.

2.5 THE DISTRIBUTED FILE SYSTEM

So far, we have mostly focused on the *processing* aspect of data-intensive processing, but it is important to recognize that without data, there is nothing to compute on. In high-performance computing (HPC) and many traditional cluster architectures, storage is viewed as a distinct and separate component from computation. Implementations vary widely, but network-attached storage (NAS) and storage area networks (SAN) are common; supercomputers often have dedicated subsystems for handling storage (separate nodes, and often even separate networks). Regardless of the details, the processing cycle remains the same at a high level: the compute nodes fetch input from storage, load the data into memory, process the data, and then write back the results (with perhaps intermediate checkpointing for long-running processes).

As dataset sizes increase, more compute capacity is required for processing. But as compute capacity grows, the link between the compute nodes and the storage becomes a bottleneck. At that point, one could invest in higher performance but more expensive networks (e.g., 10 gigabit Ethernet) or special-purpose interconnects such as InfiniBand (even more expensive). In most cases, this is not a cost-effective solution, as the price of networking equipment increases non-linearly with performance (e.g., a switch with ten times the capacity is usually more than ten times more expensive). Alternatively, one could abandon the separation of computation and storage as distinct components in a cluster. The distributed file system (DFS) that underlies MapReduce adopts exactly this approach. The Google File System (GFS) [57] supports Google’s proprietary implementation of MapReduce; in the open-source world, HDFS (Hadoop Distributed File System) is an open-source implementation of GFS that supports Hadoop. Although MapReduce doesn’t necessarily require the distributed file system, it is difficult to realize many of the advantages of the programming model without a storage substrate that behaves much like the DFS.¹⁴

Of course, distributed file systems are not new [74, 32, 7, 147, 133]. The MapReduce distributed file system builds on previous work but is specifically adapted to large-data processing workloads, and therefore departs from previous architectures in certain respects (see discussion by Ghemawat et al. [57] in the original GFS paper.). The main idea is to divide user data into blocks and replicate those blocks across the local disks of nodes in the cluster. Blocking data, of course, is not a new idea, but DFS blocks are significantly larger than block sizes in typical single-machine file systems (64 MB by default). The distributed file system adopts a master-slave architecture in which the master maintains the file namespace (metadata, directory structure, file to block mapping, location of blocks, and access permissions) and the slaves manage the actual

¹⁴However, there is evidence that existing POSIX-based distributed cluster file systems (e.g., GPFS or PVFS) can serve as a replacement for HDFS, when properly tuned or modified for MapReduce workloads [146, 6]. This, however, remains an experimental use case.

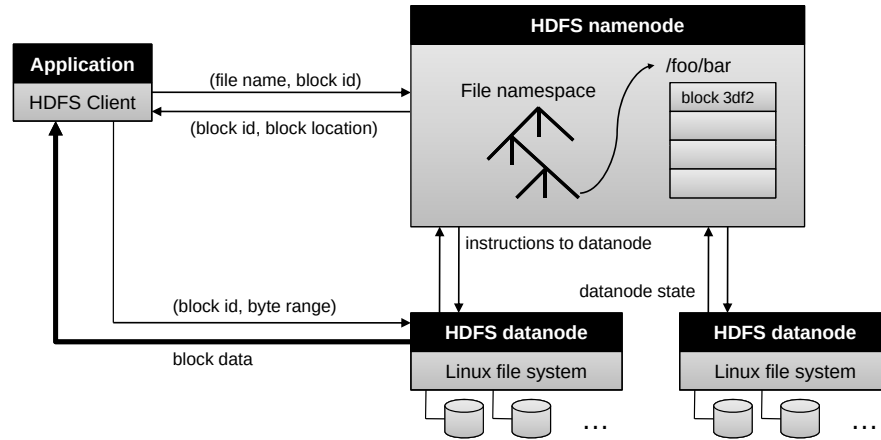


Figure 2.5: The architecture of HDFS. The namenode (master) is responsible for maintaining the file namespace and directing clients to datanodes (slaves) that actually hold data blocks containing user data.

data blocks. In GFS, the master is called the GFS master, and the slaves are called GFS chunkservers. In Hadoop, the same roles are filled by the namenode and datanodes, respectively.¹⁵ This book adopts the Hadoop terminology, although for most basic file operations GFS and HDFS work much the same way. The architecture of HDFS is shown in Figure 2.5, redrawn from a similar diagram describing GFS [57].

In HDFS, an application client wishing to read a file (or a portion thereof) must first contact the namenode to determine where the actual data is stored. In response to the client request, the namenode returns the relevant block id and the location where the block is held (i.e., which datanode). The client then contacts the datanode to retrieve the data. Blocks are themselves stored on standard single-machine file systems, so HDFS lies on top of the standard OS stack (e.g., Linux). An important feature of the design is that data is never moved through the namenode. Instead, all data transfer occurs directly between clients and datanodes; communications with the namenode only involves transfer of metadata.

By default, HDFS stores three separate copies of each data block to ensure both reliability, availability, and performance. In large clusters, the three replicas are spread across different physical racks, so HDFS is resilient towards two common failure scenarios: individual datanode crashes and failures in networking equipment that bring an entire rack offline. Replicating blocks across physical machines also increases oppor-

¹⁵To be precise, namenode and datanode may refer to physical machines in a cluster, or they may refer to daemons running on those machines providing the relevant services.

tunities to co-locate data and processing in the scheduling of MapReduce jobs, since multiple copies yield more opportunities to exploit locality. The namenode is in periodic communication with the datanodes to ensure proper replication of all the blocks: if there aren't enough replicas (e.g., due to disk or machine failures or to connectivity losses due to networking equipment failures), the namenode directs the creation of additional copies;¹⁶ if there are too many replicas (e.g., a repaired node rejoins the cluster), extra copies are discarded.

To create a new file and write data to HDFS, the application client first contacts the namenode, which updates the file namespace after checking permissions and making sure the file doesn't already exist. The namenode allocates a new block on a suitable datanode, and the application is directed to stream data directly to it. From the initial datanode, data is further propagated to additional replicas. In the most recent release of Hadoop as of this writing (release 0.20.2), files are immutable—they cannot be modified after creation. There are current plans to officially support file appends in the near future, which is a feature already present in GFS.

In summary, the HDFS namenode has the following responsibilities:

- **Namespace management.** The namenode is responsible for maintaining the file namespace, which includes metadata, directory structure, file to block mapping, location of blocks, and access permissions. These data are held in memory for fast access and all mutations are persistently logged.
- **Coordinating file operations.** The namenode directs application clients to datanodes for read operations, and allocates blocks on suitable datanodes for write operations. All data transfers occur directly between clients and datanodes. When a file is deleted, HDFS does not immediately reclaim the available physical storage; rather, blocks are lazily garbage collected.
- **Maintaining overall health of the file system.** The namenode is in periodic contact with the datanodes via heartbeat messages to ensure the integrity of the system. If the namenode observes that a data block is under-replicated (fewer copies are stored on datanodes than the desired replication factor), it will direct the creation of new replicas. Finally, the namenode is also responsible for rebalancing the file system.¹⁷ During the course of normal operations, certain datanodes may end up holding more blocks than others; rebalancing involves moving blocks from datanodes with more blocks to datanodes with fewer blocks. This leads to better load balancing and more even disk utilization.

¹⁶Note that the namenode coordinates the replication process, but data transfer occurs directly from datanode to datanode.

¹⁷In Hadoop, this is a manually-invoked process.

Since GFS and HDFS were specifically designed to support Google’s proprietary and the open-source implementation of MapReduce, respectively, they were designed with a number of assumptions about the operational environment, which in turn influenced the design of the systems. Understanding these choices is critical to designing effective MapReduce algorithms:

- The file system stores a relatively modest number of large files. The definition of “modest” varies by the size of the deployment, but in HDFS multi-gigabyte files are common (and even encouraged). There are several reasons why lots of small files are to be avoided. Since the namenode must hold all file metadata in memory, this presents an upper bound on both the number of files and blocks that can be supported.¹⁸ Large multi-block files represent a more efficient use of namenode memory than many single-block files (each of which consumes less space than a single block size). In addition, mappers in a MapReduce job use individual files as a basic unit for splitting input data. At present, there is no default mechanism in Hadoop that allows a mapper to process multiple files. As a result, mapping over many small files will yield as many map tasks as there are files. This results in two potential problems: first, the startup costs of mappers may become significant compared to the time spent actually processing input key-value pairs; second, this may result in an excessive amount of across-the-network copy operations during the “shuffle and sort” phase (recall that a MapReduce job with m mappers and r reducers involves up to $m \times r$ distinct copy operations).
- Workloads are batch oriented, dominated by long streaming reads and large sequential writes. As a result, high sustained bandwidth is more important than low latency. This exactly describes the nature of MapReduce jobs, which are batch operations on large amounts of data. Due to the common-case workload, both HDFS and GFS do not implement any form of data caching.¹⁹
- Applications are aware of the characteristics of the distributed file system. Neither HDFS nor GFS present a general POSIX-compliant API, but rather support only a subset of possible file operations. This simplifies the design of the distributed file system, and in essence pushes part of the data management onto the end application. One rationale for this decision is that each application knows best how to handle data specific to that application, for example, in terms of resolving inconsistent states and optimizing the layout of data structures.

¹⁸According to Dhruba Borthakur in a post to the Hadoop mailing list on 6/8/2008, each block in HDFS occupies about 150 bytes of memory on the namenode.

¹⁹However, since the distributed file system is built on top of a standard operating system such as Linux, there is still OS-level caching.

- The file system is deployed in an environment of cooperative users. There is no discussion of security in the original GFS paper, but HDFS explicitly assumes a datacenter environment where only authorized users have access. File permissions in HDFS are only meant to prevent unintended operations and can be easily circumvented.²⁰
- The system is built from unreliable but inexpensive commodity components. As a result, failures are the norm rather than the exception. HDFS is designed around a number of self-monitoring and self-healing mechanisms to robustly cope with common failure modes.

Finally, some discussion is necessary to understand the single-master design of HDFS and GFS. It has been demonstrated that in large-scale distributed systems, simultaneously providing consistency, availability, and partition tolerance is impossible—this is Brewer’s so-called CAP Theorem [58]. Since partitioning is unavoidable in large-data systems, the real tradeoff is between consistency and availability. A single-master design trades availability for consistency and significantly simplifies implementation. If the master (HDFS namenode or GFS master) goes down, the entire file system becomes unavailable, which trivially guarantees that the file system will never be in an inconsistent state. An alternative design might involve multiple masters that jointly manage the file namespace—such an architecture would increase availability (if one goes down, another can step in) at the cost of consistency, not to mention requiring a more complex implementation (cf. [4, 105]).

The single-master design of GFS and HDFS is a well-known weakness, since if the master goes offline, the entire file system and all MapReduce jobs running on top of it will grind to a halt. This weakness is mitigated in part by the lightweight nature of file system operations. Recall that no data is ever moved through the namenode and that all communication between clients and datanodes involve only metadata. Because of this, the namenode rarely is the bottleneck, and for the most part avoids load-induced crashes. In practice, this single point of failure is not as severe a limitation as it may appear—with diligent monitoring of the namenode, mean time between failure measured in months are not uncommon for production deployments. Furthermore, the Hadoop community is well-aware of this problem and has developed several reasonable workarounds—for example, a warm standby namenode that can be quickly switched over when the primary namenode fails. The open source environment and the fact that many organizations already depend on Hadoop for production systems virtually guarantees that more effective solutions will be developed over time.

²⁰However, there are existing plans to integrate Kerberos into Hadoop/HDFS.

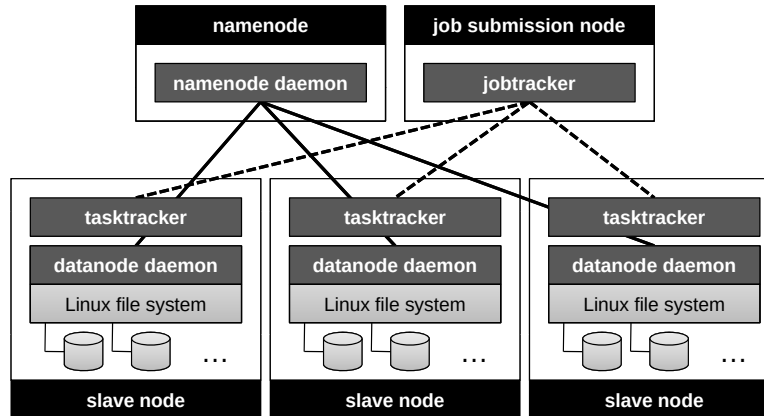


Figure 2.6: Architecture of a complete Hadoop cluster, which consists of three separate components: the HDFS master (called the namenode), the job submission node (called the jobtracker), and many slave nodes (three shown here). Each of the slave nodes runs a tasktracker for executing map and reduce tasks and a datanode daemon for serving HDFS data.

2.6 HADOOP CLUSTER ARCHITECTURE

Putting everything together, the architecture of a complete Hadoop cluster is shown in Figure 2.6. The HDFS namenode runs the namenode daemon. The job submission node runs the jobtracker, which is the single point of contact for a client wishing to execute a MapReduce job. The jobtracker monitors the progress of running MapReduce jobs and is responsible for coordinating the execution of the mappers and reducers. Typically, these services run on two separate machines, although in smaller clusters they are often co-located. The bulk of a Hadoop cluster consists of slave nodes (only three of which are shown in the figure) that run both a tasktracker, which is responsible for actually running user code, and a datanode daemon, for serving HDFS data.

A Hadoop MapReduce job is divided up into a number of map tasks and reduce tasks. Tasktrackers periodically send heartbeat messages to the jobtracker that also doubles as a vehicle for task allocation. If a tasktracker is available to run tasks (in Hadoop parlance, has empty task slots), the return acknowledgment of the tasktracker heartbeat contains task allocation information. The number of reduce tasks is equal to the number of reducers specified by the programmer. The number of map tasks, on the other hand, depends on many factors: the number of mappers specified by the programmer serves as a hint to the execution framework, but the actual number of tasks depends on both the number of input files and the number of HDFS data blocks occupied by those files. Each map task is assigned a sequence of input key-value

pairs, called an input split in Hadoop. Input splits are computed automatically and the execution framework strives to align them to HDFS block boundaries so that each map task is associated with a single data block. In scheduling map tasks, the jobtracker tries to take advantage of data locality—if possible, map tasks are scheduled on the slave node that holds the input split, so that the mapper will be processing local data. The alignment of input splits with HDFS block boundaries simplifies task scheduling. If it is not possible to run a map task on local data, it becomes necessary to stream input key-value pairs across the network. Since large clusters are organized into racks, with far greater intra-rack bandwidth than inter-rack bandwidth, the execution framework strives to at least place map tasks on a rack which has a copy of the data block.

Although conceptually in MapReduce one can think of the mapper being applied to all input key-value pairs and the reducer being applied to all values associated with the same key, actual job execution is a bit more complex. In Hadoop, mappers are Java objects with a `MAP` method (among others). A mapper object is instantiated for every map task by the tasktracker. The life-cycle of this object begins with instantiation, where a hook is provided in the API to run programmer-specified code. This means that mappers can read in “side data”, providing an opportunity to load state, static data sources, dictionaries, etc. After initialization, the `MAP` method is called (by the execution framework) on all key-value pairs in the input split. Since these method calls occur in the context of the same Java object, it is possible to preserve state across multiple input key-value pairs within the same map task—this is an important property to exploit in the design of MapReduce algorithms, as we will see in the next chapter. After all key-value pairs in the input split have been processed, the mapper object provides an opportunity to run programmer-specified termination code. This, too, will be important in the design of MapReduce algorithms.

The actual execution of reducers is similar to that of the mappers. Each reducer object is instantiated for every reduce task. The Hadoop API provides hooks for programmer-specified initialization and termination code. After initialization, for each intermediate key in the partition (defined by the partitioner), the execution framework repeatedly calls the `REDUCE` method with an intermediate key and an iterator over all values associated with that key. The programming model also guarantees that intermediate keys will be presented to the `REDUCE` method in sorted order. Since this occurs in the context of a single object, it is possible to preserve state across multiple intermediate keys (and associated values) within a single reduce task. Once again, this property is critical in the design of MapReduce algorithms and will be discussed in the next chapter.

2.7 SUMMARY

This chapter provides a basic overview of the MapReduce programming model, starting with its roots in functional programming and continuing with a description of mappers, reducers, partitioners, and combiners. Significant attention is also given to the underlying distributed file system, which is a tightly-integrated component of the MapReduce environment. Given this basic understanding, we now turn our attention to the design of MapReduce algorithms.